TFRA-YOLO: Textile Feature Refinement and Reassembly YOLO for Closed-Loop Fabric Defect Inspection on Circular Knitting Machines

Kangzhao Tan, Jiarui Zhang, Junjie Yu, Heng Wu, and Chunhua He

***Abstract*—Circular knitting machines are foundational production infrastructure in the textile industry. A fabric defect arising during production can propagate during continuous knitting and cause substantial fabric loss if it is not detected and handled promptly. To address this issue, we design and implement a cloud-edge collaborative inspection system integrating a Hall sensor, adjustable supplementary lighting, an industrial high-definition camera, an RK3566 edge terminal, a textile-specific detector, and an IoT cloud platform. When the Hall sensor indicates that the machine is operating, the edge terminal activates the camera and supplementary light. Once a defect is detected, it stops the machine through the local wired control path and reports the defect to the cloud platform for defect statistics and process improvement. The proposed TFRA-YOLO starts from YOLOv8n and combines warp–weft texture encoding (WWTE) with local texture-guided feature reassembly (LTFR) to retain directional knitted-texture cues and adaptively reconstruct the feature pyramid. On images collected from a real textile production line, TFRA-YOLO achieves 66.3% AP and 96.9% AP50, improving YOLOv8n by 7.0 and 2.8 percentage points, respectively. The proposed system provides a practical route from delayed post-production inspection toward traceable, in-process defect containment for circular knitting.**

***Index Terms*—Industrial defect detection, YOLO, deep learning, edge computing, Internet of Things.**

I. Introduction

CIRCULAR knitting machines are foundational production infrastructure in the textile industry: they continuously manufacture the knitted fabrics used in T-shirts, sportswear, and other daily garments. The latest International Textile Manufacturers Federation (ITMF) statistics report 28,000 global shipments of large circular knitting machines in 2024 [1]. Conventional inspection is commonly performed after production. Consequently, a defect arising early in production may continue propagating as the machine operates, leading to substantial fabric waste before it is discovered. An effective inspection system must therefore detect and localize defects during machine operation with sufficiently low latency to enable intervention before defects propagate into substantial fabric waste.

Single-stage YOLO detectors are particularly attractive for this setting because they jointly provide real-time inference and explicit defect localization. YOLO-based detectors have been adopted for industrial surface inspection of steel strips, steel surfaces, and large stamped parts [2]–[4], as well as for fabric-defect detection [5]–[16]. These studies establish the suitability of the YOLO route for bounding-box localization under industrial latency and resource constraints. YOLOv8n is adopted as the baseline because, on the official RK3566 benchmark, it achieves a higher inference frame rate than YOLO11n while retaining a more modern and structurally extensible architecture than YOLOv5n, making it suitable for both edge deployment and module-level modification [17].

However, two texture-specific limitations remain when a default YOLOv8n is applied to circular-knitted fabric. The dataset contains *row*, *col*, and *hole* defects, which typically appear as horizontal, vertical, and compact local disruptions of the repetitive knitted texture, respectively. Standard C2f processing can learn local defect patterns, but it does not explicitly introduce an orientation-aware inductive bias for separating these forms of texture disruption. Second, nearest-neighbor upsampling in the top-down feature pyramid reconstructs higher-resolution features with a content-independent copying rule rather than adapting reconstruction weights to local texture structure. Such content-independent reconstruction may blur or duplicate weak defect responses when thin defect boundaries are embedded in repetitive texture. Existing YOLO adaptations commonly strengthen generic feature extraction through attention, deformable or large-kernel operations, extra prediction layers, and enhanced multi-scale fusion [10], [12]–[14]. These improvements enhance generic features, but do not jointly define a directional texture decomposition and content-adaptive top-down reassembly tailored to circular-knitted textures. These observations suggest that circular-knitted defect detection requires not merely stronger generic features, but explicit directional texture encoding and content-adaptive feature reconstruction.

Based on this motivation, we develop TFRA-YOLO by adapting two key stages of the YOLOv8n detection pipeline. To address the first limitation, we introduce a warp–weft texture encoding (WWTE) module after selected C2f outputs. WWTE separately models horizontal, vertical, and compact local texture evidence, while residual fusion preserves the baseline C2f features and injects directional texture cues. To address the second limitation, we introduce a local texture-guided feature reassembly (LTFR) pyramid that replaces the two fixed nearest-neighbor top-down upsampling operations with location-specific feature reassembly. TFRA-YOLO retains the lightweight YOLOv8n framework while adding texture-aware representation at backbone encoding and content-adaptive reconstruction in the top-down feature pyramid.

Beyond detector design, practical deployment requires inference to be synchronized with the circular knitting machine

K. Tan, J. Zhang, J. Yu, and C. He are with the School of Computer Science, Guangdong University of Technology, Guangzhou 510006, China. H. Wu is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China. Corresponding author: C. He (e-mail: hechunhua@pku.edu.cn). E-mails: K. Tan (tankangzhao@mails.gdut.edu.cn), J. Zhang (zhangjiarui1@mail2.gdut.edu.cn), J. Yu (yujunjie2@mails.gdut.edu.cn), H. Wu (heng.wu@foxmail.com), and C. He (hechunhua@pku.edu.cn).

and to support low-latency local intervention. We therefore integrate TFRA-YOLO into a cloud-edge inspection system comprising rotation-synchronized image acquisition, RK3566-based edge inference, local wired stop control, and a cloud digital-twin platform. The Hall sensor provides machine-rotation information used to synchronize the inspection cycle. Once a defect is detected, the edge terminal immediately triggers the local wired stop-control path, after which the detection result is displayed for on-site operator inspection and fault diagnosis. Defect records and device states are synchronized to the cloud platform for status visualization, defect statistics, and process improvement, while the latency-critical stop-control loop remains entirely local and does not depend on cloud communication.

The main contributions of our work are as follows.

- We propose the warp–weft texture encoding (WWTE) module and integrate it after selected C2f stages of YOLOv8n. Its horizontal, vertical, and local depthwise branches explicitly encode the three characteristic forms of knitted-texture interruption while residual fusion preserves the original C2f representation.
- We design the local texture-guided feature reassembly (LTFR) pyramid for TFRA-YOLO. LTFR replaces the two fixed nearest-neighbor top-down upsampling operations with location-specific reassembly, allowing coarse semantic features to be reconstructed according to local knitted texture before multi-scale detection.
- We implement a cloud-edge inspection prototype for circular knitting machines that integrates rotation-synchronized sensing, RK3566 edge inference, local wired stop control, and cloud-based digital-twin monitoring. The design keeps the latency-critical intervention loop local while supporting centralized defect statistics and device supervision.

II. Closed-Loop Inspection Workflow

A. Production Workflow

Fig. 1 illustrates the intended operating sequence. An operator starts the circular knitting machine. The Hall sensor reports machine-rotation information to the edge side, where it is used to synchronize image acquisition and inference with the inspection cycle. The edge unit analyzes captured fabric images locally. When its decision logic identifies a defect, it immediately sends a stop request through the machine-side control interface and records the associated image and prediction. The detection result is displayed locally for operator inspection and fault diagnosis. After the operator handles the event and restarts the machine, the production cycle resumes. Device states, detection records, and diagnosis outcomes are synchronized to a cloud digital-twin platform, where they support real-time status visualization, defect-category statistics, and process-improvement analysis. Authorized users can also issue remote supervisory control actions through the platform; these actions are distinct from the local wired defect-to-stop path.

TABLE I
On-machine deployment configuration.

Item	Configuration or measured value
Edge terminal	Rockchip RK3566; Android 13
Camera	SmartSens 1.3-megapixel industrial HD global-shutter camera
Supplementary light	Adjustable illumination
Acquisition	1280×1024; nominal 30 frames/s
Installation geometry	10 cm camera-to-fabric distance; approximately 16×14 cm field of view
Inference time	45 ms per processed image (approximately 22.2 processed frames/s)
Maximum-speed coverage	38-inch diameter and 30 revolutions/min: 1.52 m/s surface speed; 6.8 cm advance per processed view
Control interface	Direct wired connection to the circular knitting machine controller
Decision-to-command latency	< 10 ms from defect decision to stop-control signal transmission
Cloud communication	MQTT-based synchronization
Cloud platform	Digital twin: live state, defect analytics, and authorized remote control
Operating mode	Designed for continuous 24/7 inspection

B. On-Machine Deployment Configuration

The deployed edge terminal uses a Rockchip RK3566 processor running Android 13. Images are acquired with a SmartSens 1.3-megapixel industrial high-definition global-shutter camera at 1280×1024 resolution and a nominal 30 frames/s. Adjustable supplementary lighting is used to stabilize the local imaging condition. The camera is installed 10 cm from the fabric, providing an approximately 16×14 cm field of view. On the deployed terminal, the reported inference time is 45 ms per processed image (approximately 22.2 processed frames/s). This rate is sufficient for the maximum stated machine condition: a 38-inch circular knitting machine at 30 revolutions/min has a circumference of approximately 3.03 m and a fabric-surface speed of approximately 1.52 m/s. Consequently, the surface advances approximately 6.8 cm during one 45-ms inference interval, which remains below the 14-cm shorter side of the field of view and yields at least 7.2 cm nominal overlap between adjacent processed views. The edge terminal is connected to the circular knitting machine control system by a direct wired interface; the measured time from a defect decision to transmission of the stop-control signal is below 10 ms. The edge terminal synchronizes non-latency-critical status and defect records with the cloud IoT platform through MQTT. The system is designed to support continuous 24/7 inspection. Table I summarizes the configuration.

C. Platform-Aware Baseline Selection and NPU Deployment

The choice of YOLOv8n as the baseline is driven by deployment compatibility rather than by version chronology alone. The RKNN Model Zoo provides tested RK3566 NPU examples and single-core INT8 throughput figures for several YOLO families at 640×640 input [17]. In the current official benchmark, YOLOv8n reaches 34.0 frames/s, exceeding the listed newer YOLOv10n (20.7 frames/s) and YOLO11n (20.6 frames/s). The legacy YOLOv5n and YOLOv6n entries

are faster (39.7 and 48.8 frames/s, respectively), which directly demonstrates that a newer YOLO version is not automatically a better NPU choice. We therefore select YOLOv8n as a high-throughput baseline with tested RK3566 NPU examples in the official Model Zoo and an extensible structure for textile-specific modification, rather than claiming that it is the absolute fastest model in the official benchmark. The benchmark values are used only for platform-aware model selection: they do not substitute for measuring the final customized detector on the deployed application.

The training and deployment models use different representations. Training produces a PyTorch checkpoint, which is exported at 640×640 through ONNX and then converted to an RKNN model for the RK3566 NPU. We use the official RKNN-Toolkit2 conversion workflow, which targets Rockchip NPUs and supports model conversion, quantization, inference, and performance evaluation [18]. Integer-only quantization is a well-established route to reduce inference cost on constrained hardware [19]. In the deployment path, post-training INT8 quantization is applied during ONNX-to-RKNN conversion with representative training images used for calibration; the resulting RKNN artifact is loaded by the Android edge application for NPU execution. This workflow is summarized as *PyTorch checkpoint* $\rightarrow$ *ONNX* $\rightarrow$ *INT8 RKNN* $\rightarrow$ *RK3566 NPU*. The reported 45-ms edge inference time in Table I corresponds to the deployed application, not PyTorch inference on the training workstation.

Fig. 2 provides the actual on-machine installation. The wide view shows the circular knitting machine on the production floor. The upper inset highlights the camera directed at the knitting region, and the lower inset shows the edge-side touch display during inspection. Together with the workflow in Fig. 1, this installation evidence distinguishes the work from an offline-only image experiment.

Fig. 3 shows the corresponding cloud IoT platform. The platform provides a portal view, defect-count trend statistics, device status and remote configuration, and a review page for uploaded defect records. These cloud functions support supervisory operation and process analysis; they are separated from the latency-critical local wired stop path.

The design follows the edge-intelligence principle that time-sensitive perception and action remain near the physical process, while selected evidence can be retained centrally [20]. The cloud side is organized as a manufacturing digital-twin service for live state visualization, statistics, and authorized supervision [21], [22]. Here, the control consequence is more direct: the event is escalated to a worker, not accepted as a final autonomous verdict. This provides a pragmatic route for handling rare defects while retaining the records needed to identify recurring defect patterns and process anomalies.

D. Operational Boundaries

The workflow separates three stages: (i) whether machine rotation enables inspection, (ii) whether the detector produces a defect decision and initiates the local stop path, and (iii) how an operator diagnoses and disposes of the stopped-machine event. The reported recognition experiments validate only the second stage on a held-out image split. The deployed configuration establishes geometric image coverage at the stated maximum machine speed, the camera, edge terminal, wired control interface, and decision-to-command latency, but does not yet quantify physical machine-stop time, false-stop rate on defect-free fabric, frame-delivery or drop statistics during continuous operation, or operator-diagnosis completion rate. These measurements should be reported from a controlled site trial before submission.

III. TFRA-YOLO

A. Overview

TFRA-YOLO is built from the YOLOv8n detector [23]. As shown in Fig. 4, the backbone retains the original resolution hierarchy, but replaces the C2f output refinement at four stages with C2f–WWTE. In the head, the two top-down nearest-neighbor upsampling operations are replaced with LTFR. The standard concatenation, C2f fusion, bottom-up path, and three-scale detection head are retained. The detection scales are P_3, P_4, and P_5, corresponding to 80×80, 40×40, and 20×20 feature maps for a 640×640 input.

B. Warp–Weft Texture Encoding

The visual cue of a defect in knitted fabric is usually not an isolated object boundary. Instead, it is a local violation of the repetitive loop texture: a *row* defect typically interrupts continuity across the width direction, a *col* defect typically interrupts continuity along the height direction, and a *hole* produces a compact local discontinuity. A standard C2f output preserves multi-path features and can learn such local patterns, but it does not explicitly introduce a directional decomposition for these three forms of texture interruption. The term "warp–weft" is retained as the internal WWTE name because it denotes two orthogonal texture directions. In this circular-knit application, it does *not* imply that the fabric is woven; operationally, the branches are interpreted as horizontal and vertical texture encoders.

WWTE is inserted *after* the selected C2f outputs rather than replacing the C2f gradient-flow structure. Thus, the original C2f features remain available through the residual pathway, while WWTE learns a residual texture correction at multiple backbone depths. For a C2f output $X \in \mathbb{R}^{C \times H \times W}$, the module applies depthwise horizontal, vertical, and local branches and then fuses them:

$$F(X) = \phi\left([\mathrm{DW}_{1\times7}(X), \mathrm{DW}_{7\times1}(X), \mathrm{DW}_{3\times3}(X)]\right), \quad (1)$$

where $[\cdot]$ denotes channel concatenation and ϕ is a 1×1 convolution with normalization and activation. The 1×7 branch aggregates evidence along a horizontal trace without first mixing channels, whereas the 7×1 branch performs the analogous vertical aggregation. The 3×3 branch retains compact local evidence required for holes and short texture breaks. Depthwise filtering keeps the long directional receptive fields inexpensive; the following pointwise fusion learns how much horizontal, vertical, and local evidence should be retained at

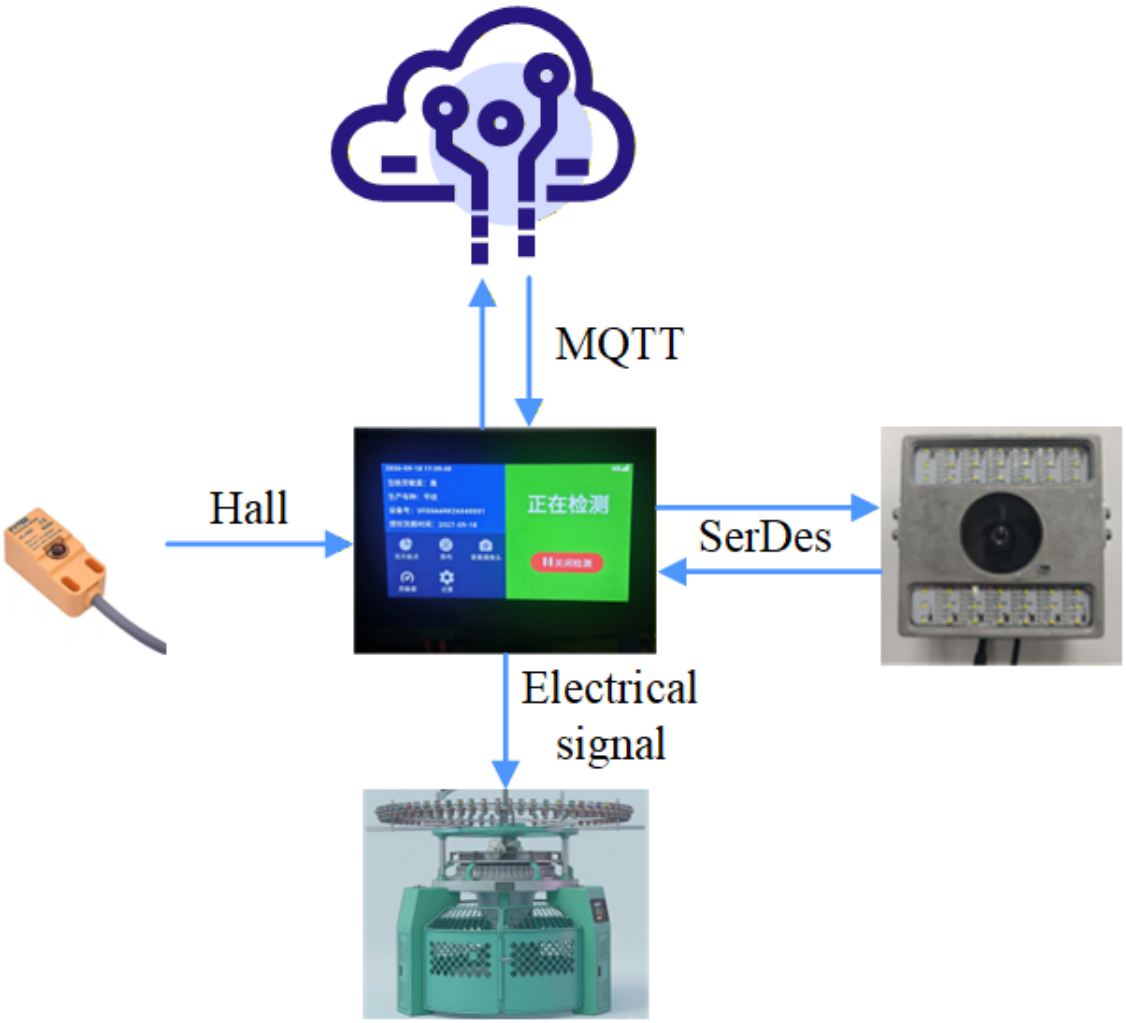


Fig. 1. System overview of the deployed circular-knitting inspection setup. A Hall sensor reports the machine rotation state to the edge terminal; the industrial camera sends images through a SerDes link; the edge terminal issues the wired electrical stop-control signal to the machine; and the cloud platform exchanges status and inspection records through MQTT.

each feature channel. Because input and output channels match in the present implementation, the module uses residual fusion:

$$Y = X + F(X). \tag{2}$$

Equation (2) is important for an edge-deployed adaptation: the C2f feature X remains directly available if a directional response is unhelpful, while $F(X)$ adds only the learned, task-specific texture cue. The design is related to, but not equivalent to, asymmetric convolution blocks [24]: it uses depthwise directional paths, three-way concatenation, and a learned pointwise fusion within the existing C2f structure. Its intended effect is therefore selective sensitivity to elongated horizontal and vertical discontinuities without sacrificing the compact context used for holes.

C. Local Texture-Guided Feature Reassembly

The features enriched by WWTE must next be passed from a coarse semantic level to a finer detection level. Fixed nearest-neighbor upsampling copies a source value to a 2×2 region according to a content-independent rule; its reconstruction weights do not adapt to local texture structure. This can be suboptimal in the top-down feature-pyramid transitions [25], where coarse semantic features must be aligned with high-resolution texture evidence and narrow defect boundaries may occupy only a small fraction of the local region. LTFR therefore replaces only these two fixed upsampling operations; the original concatenation, C2f fusion, bottom-up path, and detection head remain unchanged.

LTFR instantiates the content-aware reassembly principle of CARAFE [26] in this restricted pyramid role. Given $X \in \mathbb{R}^{C\times H\times W}$, a 1×1 compressor first reduces the kernel-prediction feature to $C_r = \max(C/4, 16)$ channels. A 3×3 encoder then produces s^2k^2 kernel logits, where the scale factor is $s = 2$ and the reassembly kernel size is $k = 5$. Pixel shuffle expands these logits onto the upsampled grid, and softmax yields a distinct normalized 5×5 kernel $K_{i,j}$ at each output location:

$$K_{i,j} = \mathrm{softmax}\left(\mathrm{PS}\left(E_{3\times3}(E_{1\times1}(X))\right)_{i,j}\right). \tag{3}$$

Here, $E_{1\times1}$ and $E_{3\times3}$ denote the compression and kernel-encoding layers, respectively, and PS denotes pixel shuffle.

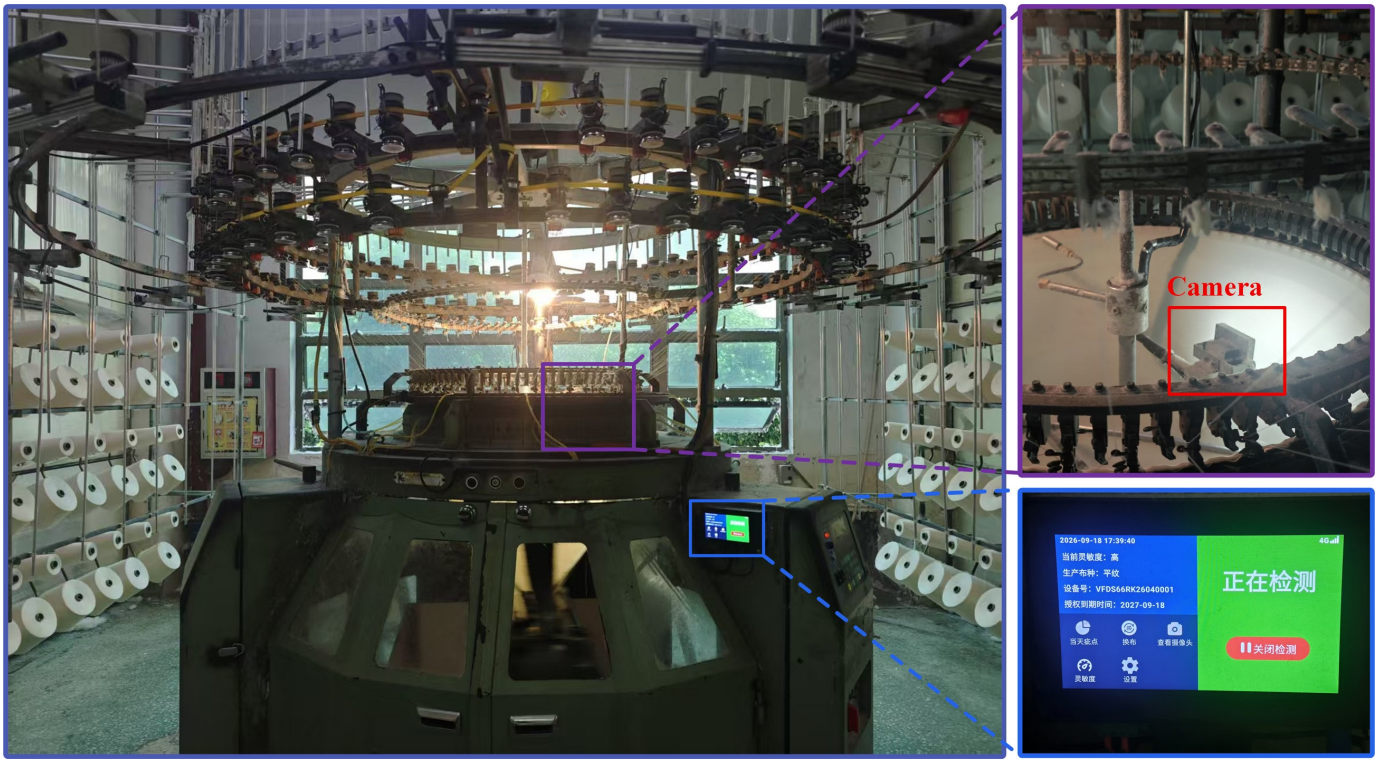


Fig. 2. Actual production-line installation on the circular knitting machine. The upper inset highlights the camera location; the lower inset shows the edge-side touch display while inspection is active. Colored callouts and insets are retained from the installation record.

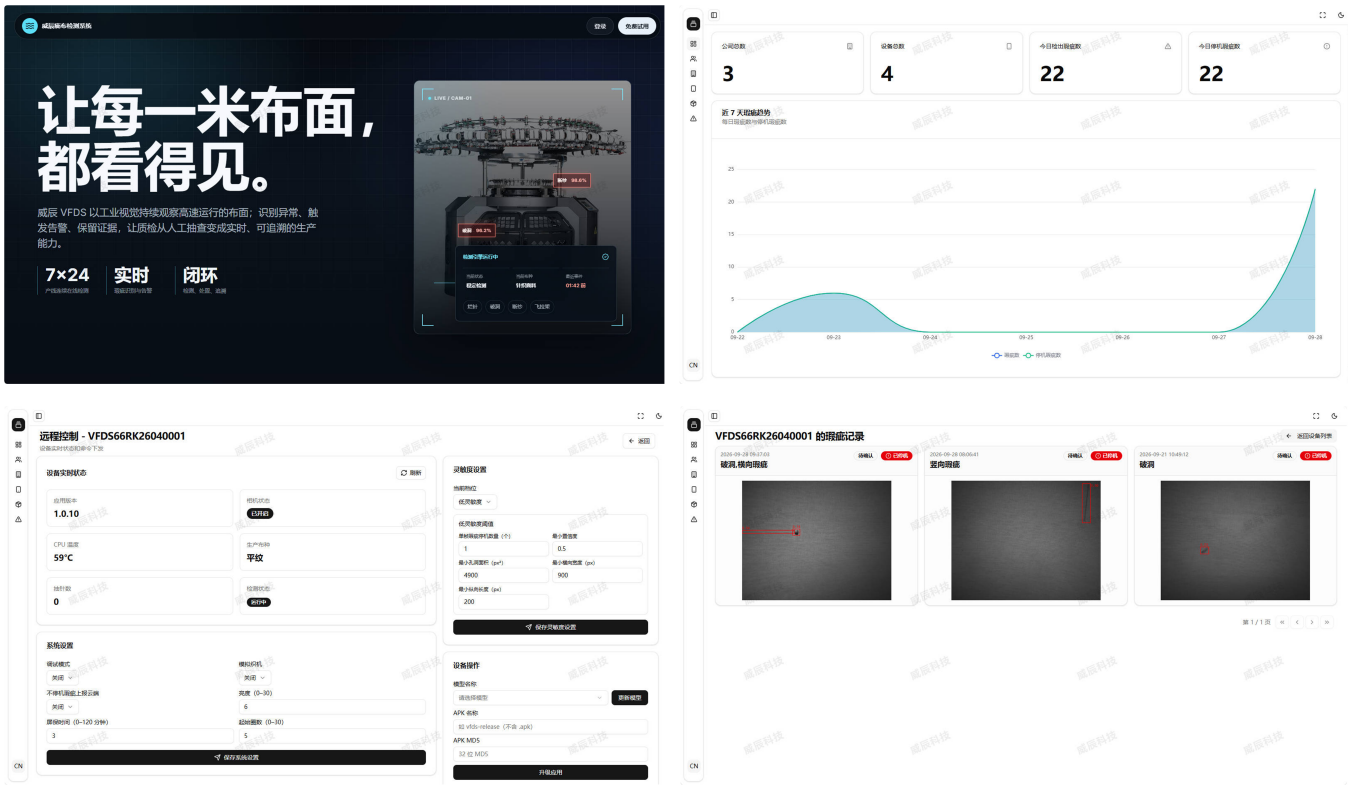


Fig. 3. Cloud IoT platform for the deployed inspection system. The four panels show the platform portal, defect-count trend dashboard, device-state and remote-configuration interface, and uploaded defect-record review interface, respectively. Cloud communication supports status visualization, statistics, and authorized supervision, whereas the defect-to-stop control path remains local and wired.

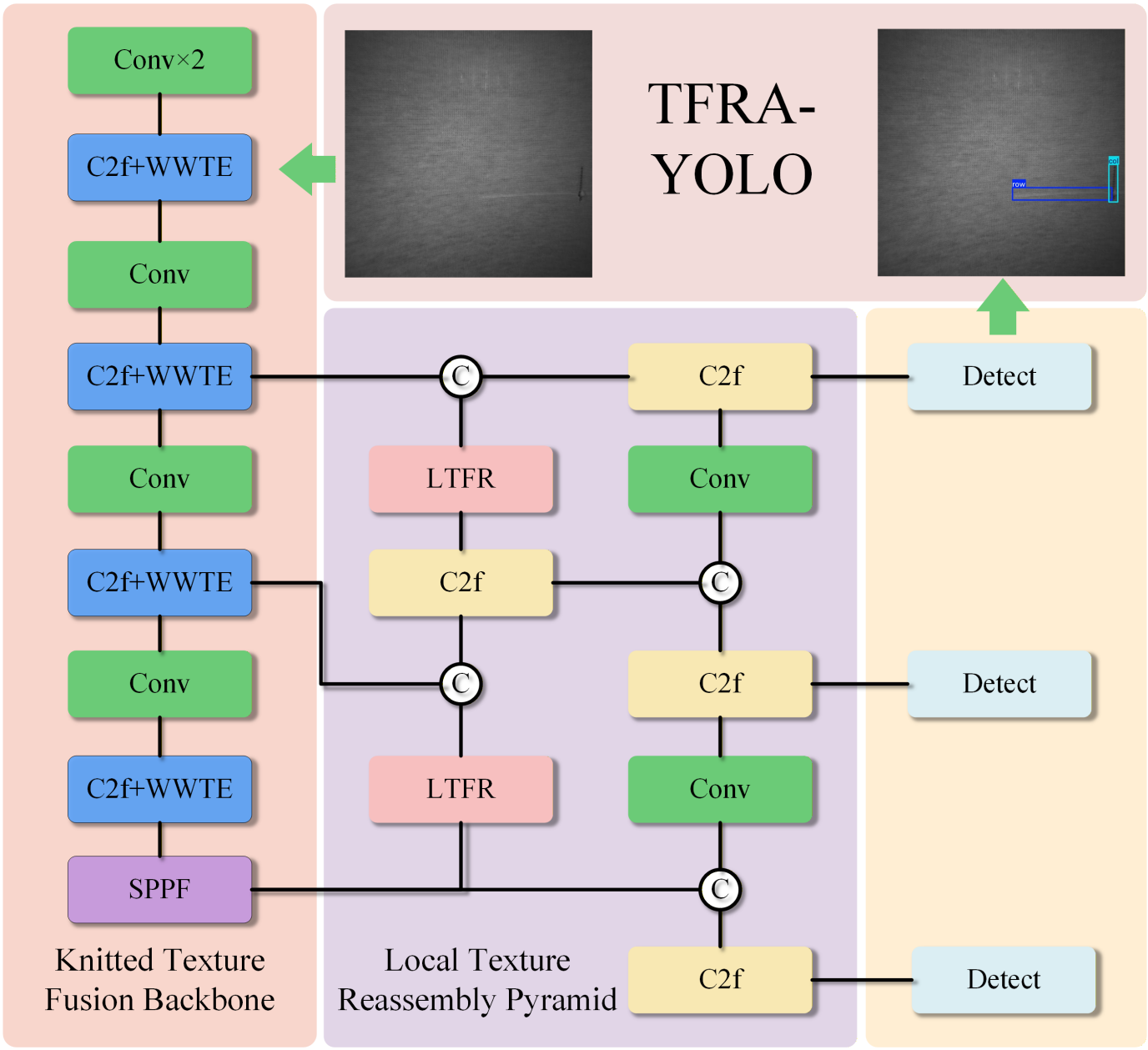


Fig. 4. Architecture of TFRA-YOLO. The Knitted Texture Fusion Backbone uses C2f–WWTE blocks to encode horizontal, vertical, and local texture cues. The Local Texture Reassembly Pyramid replaces the two top-down fixed upsampling operations with LTFR, while the bottom-up path and three-scale detection head follow YOLOv8n.

The 25 weights of each $K_{i,j}$ sum to one. For each output position, a 5×5 neighborhood $\mathcal{N}_{i,j}(X)$ is unfolded from X, spatially aligned with the enlarged grid, and reassembled as

$$Y_{i,j} = \sum_{u=1}^{25} K_{i,j,u} \mathcal{N}_{i,j,u}(X). \tag{4}$$

Unlike a spatially shared interpolation kernel, Eq. (4) allows a location near a thin defect or a texture boundary to select a different local mixture from a location within regular fabric. The channel-compression stage bounds the overhead of predicting these kernels, while the 5×5 neighborhood provides more local support than a copied source pixel. LTFR is not claimed as a new generic reassembly primitive; its contribution here is the lightweight placement of input-dependent reassembly at the two texture-critical top-down transitions of YOLOv8n.

WWTE and LTFR have complementary roles. WWTE first creates direction-sensitive cues in the backbone; LTFR then propagates and aligns those cues during top-down reconstruction instead of indiscriminately replicating them. The component ablation in Table V tests this design: WWTE produces the larger isolated gain, LTFR still improves the baseline on its own, and their combination yields the best result. Figs. 5 and 6 detail the two modules.

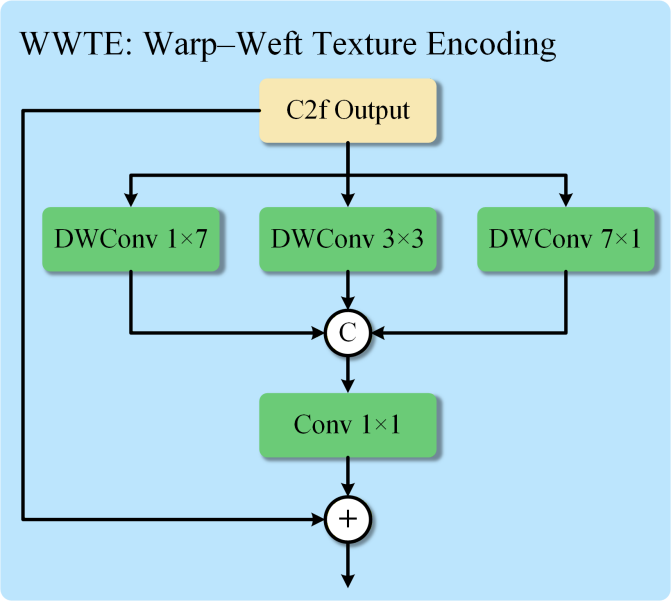


Fig. 5. WWTE structure. A C2f feature is processed by horizontal, vertical, and local depthwise branches, followed by channel concatenation, 1×1 fusion, and residual addition.

TABLE II
DATASET SPLIT SUPPLIED FOR THE CURRENT DRAFT.

Split	Images	Instances	Row	Col	Hole
Train	282	329	174	75	80
Validation	40	48	24	13	11
Test	81	91	60	12	19

IV. Experimental Protocol

A. Dataset

The data were collected from a real textile production line. The three object categories retain their engineering labels *row*, *col*, and *hole*; *row* and *col* typically manifest as horizontal and vertical texture disruptions, respectively. Table II summarizes the provided image-level split and instance counts. The test set contains 81 images with 91 labeled defects. This small, imbalanced test set makes every retained result useful for engineering comparison but is insufficient by itself to support a broad generalization claim. A future submission should state whether the split is grouped by production roll, machine, material, or time interval, and should add an independent production-period test set if available.

B. Training and Evaluation

Table III summarizes the implementation protocol used for the Ultralytics experiments. TFRA-YOLO and its direct YOLOv8n baseline are initialized from `yolov8n.pt`; the remaining comparison models use the available official pretrained initialization in their respective repositories. Evaluation is conducted on the held-out test split with a maximum of 16 detections per image.

C. Evaluation Metrics

We use the COCO detection protocol [27]. For a predicted box B_p and a ground-truth box B_g, intersection over union (IoU) is defined as $\mathrm{IoU}(B_p, B_g) = |B_p \cap B_g|/|B_p \cup B_g|$. At an IoU threshold t, a prediction is counted as a true positive only when it has the correct class and is matched one-to-one to an as-yet unmatched ground-truth instance; all remaining predictions are false positives and unmatched ground-truth instances are false negatives. For each class c, $\mathrm{AP}_c(t)$ is the area under its interpolated precision–recall curve. With $C = 3$ classes, the reported AP and AP50 are

$$\mathrm{AP} = \frac{1}{10C}\sum_{c=1}^{C}\sum_{t\in\mathcal{T}}\mathrm{AP}_c(t),$$
$$\mathrm{AP50} = \frac{1}{C}\sum_{c=1}^{C}\mathrm{AP}_c(0.50), \tag{5}$$

where $\mathcal{T} = \{0.50, 0.55, \ldots, 0.95\}$. Thus, AP measures localization and classification quality across increasingly strict overlap thresholds, whereas AP50 emphasizes detection at IoU $= 0.50$. Both are written as percentages in the tables. We additionally report trainable parameters and FLOPs at $[1, 3, 640, 640]$ to characterize model scale and computational cost; these are not substitutes for measured edge-device latency.

For each model, the retained best-validation checkpoint is evaluated once on the same held-out test split. FLOPs and parameter counts are reported at $[1, 3, 640, 640]$. The values are suitable for a coarse efficiency comparison, but cross-framework FLOP accounting can omit unsupported operators; no cross-framework latency claim is made. DETR, Deformable DETR, Faster R-CNN, and RT-DETRv2 follow their respective repositories and pretrained backbones [28]–[31].

V. Results and Discussion

A. Comparison With Detectors

Table IV compares TFRA-YOLO with the retained baselines. TFRA-YOLO achieves 66.3% AP and 96.9% AP50. Relative to its direct YOLOv8n baseline, AP improves by 7.0 points and AP50 by 2.8 points for 0.38 M additional parameters and 0.8 G additional FLOPs. Among the compact YOLO variants evaluated here, TFRA-YOLO has the highest AP. RT-DETR-R18 obtains 67.7% AP, 1.4 points higher than TFRA-YOLO, but requires 20.09 M parameters and 60.72 G FLOPs. This is a useful trade-off rather than a claim that TFRA-YOLO dominates every detector.

Fig. 7 visualizes the parameter- and operation-accuracy trade-off without shifting any data coordinates. The position of TFRA-YOLO in the low-parameter region is consistent with the tabulated result. Because the horizontal axes are logarithmic, the visual is intended to show scale rather than imply a linear performance-efficiency relationship.

B. Ablation

Table V isolates the two modifications within the same YOLOv8n family. WWTE contributes the larger isolated

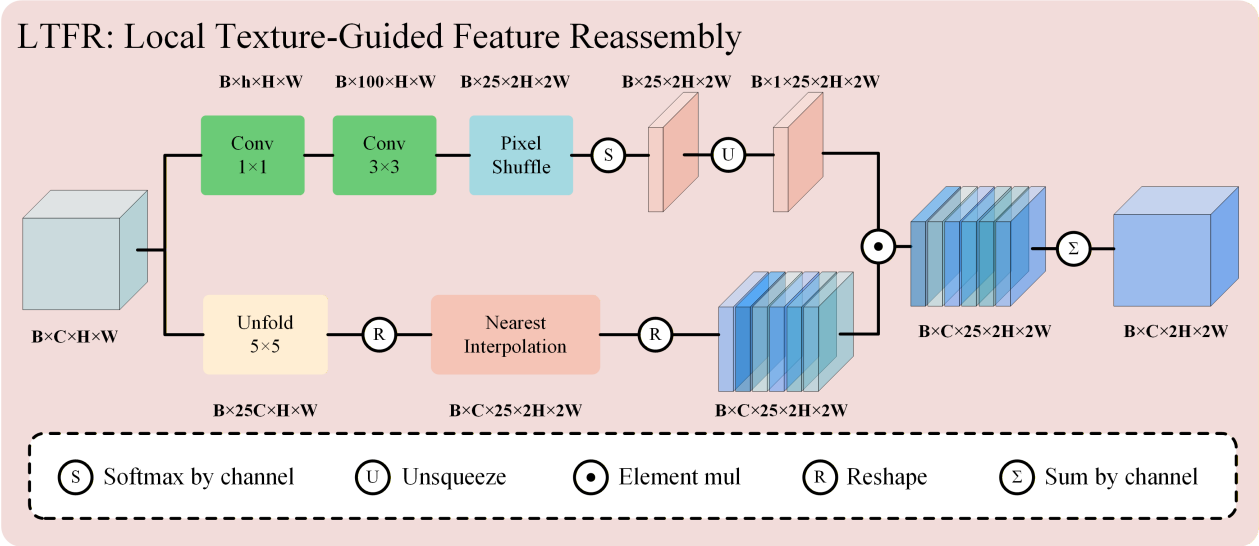


Fig. 6. LTFR structure. The upper path predicts location-specific 5×5 reassembly kernels, and the lower path unfolds and aligns local feature patches. Their element-wise product is summed along the kernel dimension to produce a $2\times$ upsampled feature.

TABLE III
IMPLEMENTATION PROTOCOL AND TRAINING HYPERPARAMETERS.

Item	Specification
Training hardware	NVIDIA GeForce RTX 3090 GPU (24 GB)
Software	Ubuntu 24.04, Python 3.10.20, PyTorch 2.4.1+cu124, and Ultralytics 8.4.140
Model initialization	TFRA-YOLO and YOLOv8n: `yolov8n.pt`; other baselines: the available official pretrained checkpoint in the corresponding repository
Input resolution	640×640
Batch size / data workers	16 / 8
Maximum epochs / early stopping patience	300 / 80 epochs
Optimizer	Ultralytics automatic optimizer selection
Data augmentation	Standard Ultralytics geometric and photometric augmentations; no augmentation hyperparameter was manually overridden
Checkpoint retained for testing	Framework-selected best validation checkpoint
Test protocol	Held-out test split, maximum 16 detections per image

TABLE IV
TEST-SET COMPARISON ON 81 IMAGES AND 91 INSTANCES. AP IS COCO AP@[.50:.95]. ALL VALUES ARE PERCENTAGES EXCEPT PARAMETERS AND FLOPS.

Model	Params (M)	FLOPs (G)	AP50 (%)	AP (%)
YOLOv5n†	2.50	7.10	93.1	58.4
YOLOv8n	3.01	8.10	94.1	59.3
YOLO11n	2.58	6.40	95.0	59.6
YOLO12n	2.56	7.30	94.3	60.9
YOLO26n	2.38	5.30	96.5	61.4
RT-DETR-R18	20.09	60.72	95.2	**67.7**
DETR-R50	41.28	77.25	96.4	53.6
Deformable DETR-R50	39.82	158.66	96.8	64.3
Faster R-CNN-R50	41.13	181.45	96.1	64.7
TFRA-YOLO	3.39	8.90	**96.9**	66.3

†The retained YOLOv5n checkpoint is the legacy YOLOv5n architecture, not the Ultralytics YOLOv5u variant.

improvement (+4.8 AP), while LTFR contributes +1.6 AP. Combining both gives 66.3% AP, which is 0.6 points above the arithmetic sum of the isolated gains. This indicates that the texture representation and adaptive reassembly are complementary: WWTE supplies direction-specific texture cues, whereas LTFR transfers them through the top-down path.

C. Per-Class and Qualitative Results

The TFRA-YOLO checkpoint obtains 61.7%, 73.8%, and 63.4% AP for *row* (horizontal), *col* (vertical), and *hole* de-

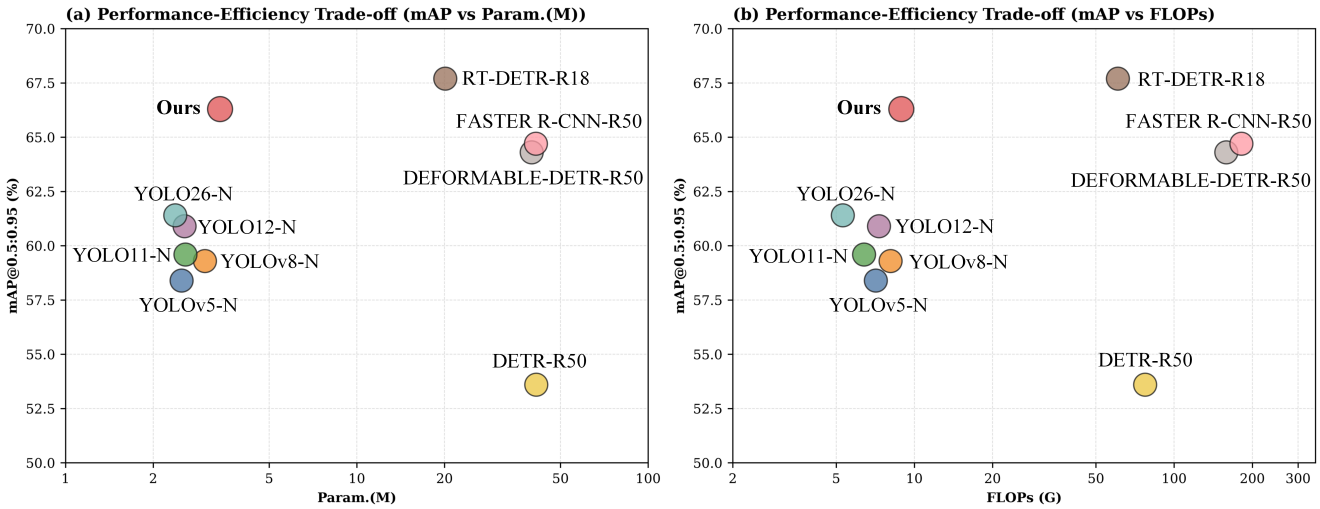


Fig. 7. Accuracy-efficiency trade-off of the retained test results. Both horizontal axes use a logarithmic scale. Marker coordinates are the exact values in Table IV; the plot does not contain manual coordinate compression.

TABLE V
ABLATION ON THE COMMON HELD-OUT TEST SPLIT USING THE SAME YOLOV8N TRAINING RECIPE.

Variant	WWTE	LTFR	Params	FLOPs	AP50	AP
YOLOv8n	–	–	3.01	8.1	94.1	59.3
+ WWTE	✓	–	3.28	8.8	94.2	64.1
+ LTFR	–	✓	3.11	8.2	94.3	60.9
TFRA-YOLO	✓	✓	3.39	8.9	96.9	66.3

fects, respectively. The notably high *col*-defect AP should be interpreted with care because the test category has only 12 instances. Fig. 8 presents seven selected images and the available standardized renderings. The visual comparison is illustrative rather than a replacement for the full quantitative test set. It is particularly useful for examining thin *col*-type defects, long *row*-type defects, and holes against repetitive background texture.

Fig. 9 further compares the activation visualizations produced by the official Ultralytics visualization routine for the YOLOv8n baseline and TFRA-YOLO. Each row corresponds to the same test image, with the original image, baseline visualization, and TFRA-YOLO visualization arranged from left to right. These maps are used only as qualitative evidence of where the detectors respond; the AP results in Tables IV and V remain the basis for the quantitative comparison.

D. Limitations and Submission-Ready Validation

The present evidence supports a promising lightweight detector on a real production-line image set, but it does not yet establish production reliability. First, the test split is small and category-imbalanced. Second, FLOPs are not a substitute for end-to-end edge latency, especially when image capture, preprocessing, postprocessing, control communication, and physical stopping are part of the operational loop. Finally, operator diagnosis records can support defect statistics and process analysis, but their completeness and operational value have not yet been measured. These points define a concrete validation plan, rather than weaknesses that should be concealed.

VI. CONCLUSION

This work presents a closed-loop framework for defect inspection on a circular knitting machine and TFRA-YOLO, a compact direction- and content-adaptive detector. The workflow couples rotation-aware edge inspection, automatic local wired stop signaling, on-site operator inspection and fault diagnosis, and a cloud digital twin for live status visualization, defect statistics, process improvement, and authorized remote supervisory control. The detector combines WWTE for horizontal, vertical, and local knitted-texture encoding with LTFR for adaptive feature-pyramid reconstruction. On the retained test split, TFRA-YOLO improves YOLOv8n from 59.3% to 66.3% AP at 3.39 M parameters and 8.9 G FLOPs. The next step is not merely more offline accuracy: it is a disciplined site evaluation of sensing-to-stop latency, false-stop behavior, operator-diagnosis workflow, process-analysis value, and grouped generalization across production conditions.

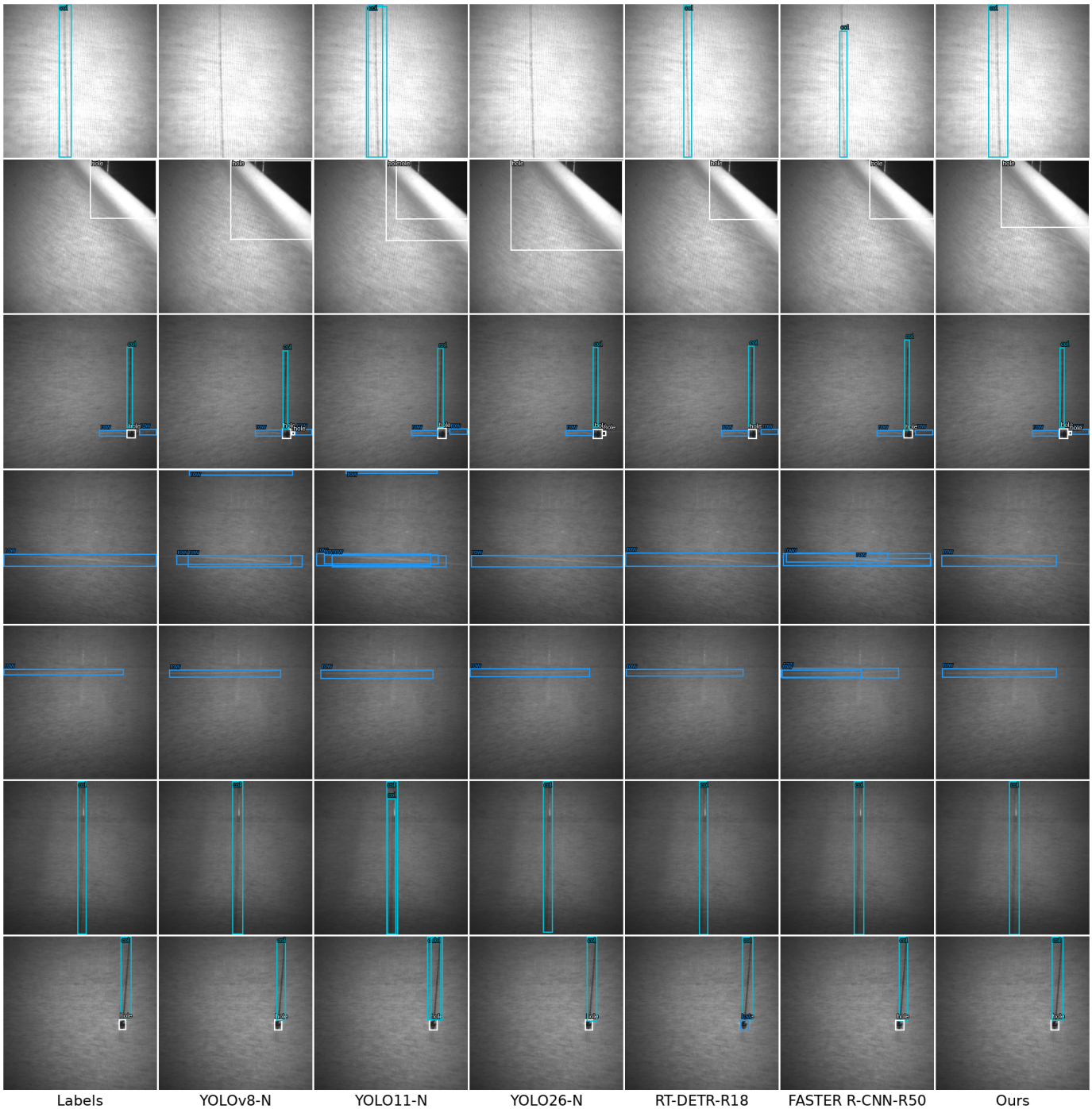


Fig. 8. Qualitative comparison on selected test images. Columns are labels, YOLOv8n, YOLO11n, YOLO26n, RT-DETR-R18, Faster R-CNN-R50, and TFRA-YOLO. The figure uses a common renderer for visual consistency; it should be updated after finalizing the exact sample-selection protocol and display confidence threshold.

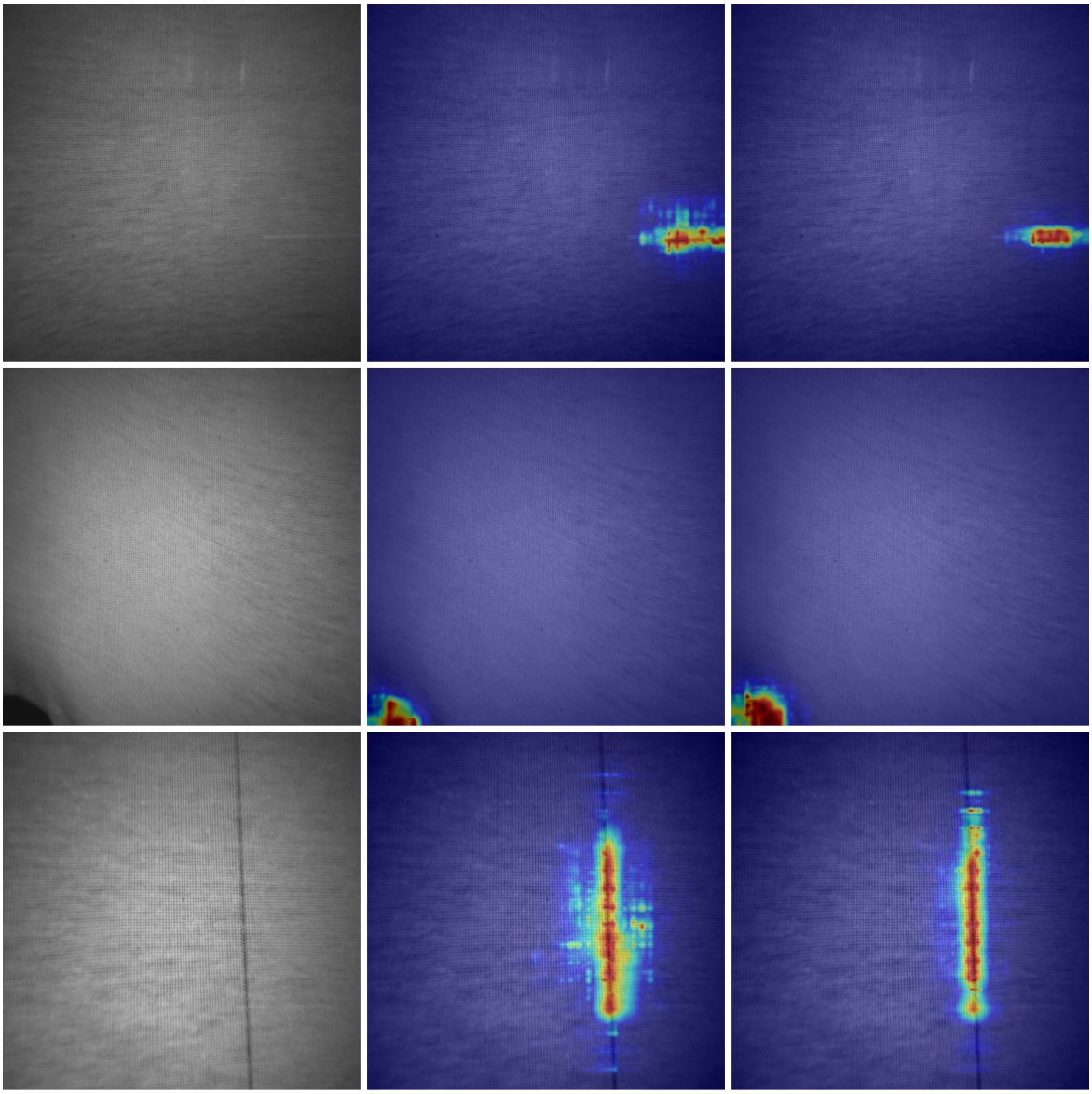


Fig. 9. Qualitative activation visualization on three selected test images. The left, middle, and right columns show the original images, YOLOv8n visualizations, and TFRA-YOLO visualizations, respectively. Each visualization is generated with the same official Ultralytics routine.

REFERENCES

[1] International Textile Manufacturers Federation, "International textile machinery shipment statistics (ITMSS), vol. 47/2024," 2025, [Online]. Available: https://www.itmf.org/publications/statistics-publications/international-textile-machinery-shipment-statistics-itmss-2024.

[2] S. Gao, M. Chu, and L. Zhang, "A detection network for small defects of steel surface based on YOLOv7," *Digital Signal Processing*, vol. 149, p. 104484, 2024.

[3] C. Li, A. Xu, Q. Zhang, and Y. Cai, "Steel surface defect detection method based on improved YOLOX," *IEEE Access*, vol. 12, pp. 37 643–37 652, 2024.

[4] X. Dong, C. Zhang, J. Wang, Y. Chen, and D. Wang, "Real-time detection of surface cracking defects for large-sized stamped parts," *Computers in Industry*, vol. 159, p. 104105, 2024.

[5] X. Yue, Q. Wang, L. He, Y. Li, and D. Tang, "Research on tiny target detection technology of fabric defects based on improved YOLO," *Applied Sciences*, vol. 12, no. 13, p. 6823, 2022.

[6] K. Yu, W. Lyu, X. Yu, Q. Guo, W. Xu, and L. Zhang, "FA-YOLO: A high-precision and efficient method for fabric defect detection in textile industry," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E107.A, no. 6, pp. 890–898, 2024.

[7] H. Yang, J. Jing, Z. Wang *et al.*, "YOLOV4-TinyS: A new convolutional neural architecture for real-time detection of fabric defects in edge devices," *Text. Res. J.*, vol. 94, pp. 49–68, 2024.

[8] X. Kang and J. Li, "AYOLOv7-tiny: Towards efficient defect detection in solid color circular weft fabric," *Text. Res. J.*, vol. 94, pp. 225–245, 2024.

[9] X. Luo, Z. Cheng, Q. Ni *et al.*, "Defect detection algorithm for fabric based on deformable convolutional network," *Textile Research Journal*, vol. 93, no. 9–10, pp. 2342–2354, 2023.

[10] C. Chen, Q. Zhou, S. Li, D. Luo, G. Tan *et al.*, "Fabric defect detection algorithm based on improved YOLOv8," *Textile Research Journal*, vol. 95, no. 3–4, pp. 235–251, 2025.

[11] C. Chen, Q. Zhou, L. Xiao *et al.*, "LWFDD-YOLO: A lightweight defect detection algorithm based on improved YOLOv8," *Textile Research Journal*, vol. 95, no. 9–10, pp. 1125–1142, 2025.

[12] L. Zhou, B. Ma, Y. Dong, Z. Yin, and F. Lu, "DCFE-YOLO: A novel fabric defect detection method," *PLOS ONE*, vol. 20, no. 1, p. e0314525, 2025.

[13] Z. Gao, S. Zhang, J. Su, B. Li, J. Wang, and Z. Song, "Research on improved YOLOv8 fabric surface defect detection algorithm," *Textile Research Journal*, vol. 96, no. 17–18, 2026.

[14] S. Xu, C. Wan, W. Feng, J. Li, and J. Gao, "YOLO-DLL: An effective detection method for fabric defects," *Textile Research Journal*, 2025.

[15] T. Wang *et al.*, "SRSE-YOLO: An improved fabric defect detection method for YOLOv8," *Textile Research Journal*, 2025.

[16] Z. Zhang, T. Huang, W. Zhang *et al.*, "DGHR-YOLO: Fabric defect detection based on high-level screening-feature pyramid networks and deformable convolution," *Nondestructive Testing and Evaluation*, 2025.

[17] Rockchip, "Rknn model zoo," [Online]. Available: https://github.com/airockchip/rknn_model_zoo, 2026, accessed: 2026-09-27.

[18] ——, "Rknn toolkit2," [Online]. Available: https://github.com/airockchip/rknn-toolkit2, 2026, accessed: 2026-09-27.

[19] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.

[20] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[21] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019.

[22] W. Kritzinger, G. Karner, C. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.

[23] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.

[24] X. Ding, Y. Guo, G. Ding, and J. Han, "Acnet: Strengthening the kernel skeletons for powerful cnn via asymmetric convolution blocks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1911–1920.

[25] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2117–2125.

[26] J. Wang, K. Chen, R. Xu, Z. Liu, C. C. Loy, and D. Lin, "Carafe: Content-aware reassembly of features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3007–3016.

[27] T.-Y. Lin, M. Maire, S. Belongie *et al.*, "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.

[28] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.

[29] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.

[30] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.

[31] W. Lv, Y. Xu, Y. Yang, G. Li, J. Wang, L. Chen *et al.*, "Rt-detrv2: Improved baseline with bag-of-freebies for real-time detection transformer," *arXiv preprint arXiv:2407.17140*, 2024.